Dynamic Dungeon Procedural Tilemap Level Generation Tool

October 2026

By

Ellison Dean

Student number 202319745

Word count: 1930

Contents

1. Project background and purpose 3
1.1. Introduction 3
1.2. Objectives 3
Primary Objectives 3
Secondary Objectives 3
1.3. Scope 3
1.4. Deliverables 4
1.5. Constraints 4
1.6. Assumptions 4
2. Project rationale and operation 5
2.1. Project benefits 5
2.2. Project operation 5
2.3. Options 5
2.4. Risk analysis and mitigation 6
2.5. Resources required 7
2.6. Ethical and legal considerations 7
2.7. Commercial considerations 7
3. Project methodology and outcomes 8
3.1. Initial project plan 8
3.1.1. Tasks and milestones 8
3.2. Schedule Gantt chart 10
3.3. Project control 13
3.4. Project evaluation 13

1. Project background and purpose

1.1. Introduction

This project is a tool designed to aid game developers in implementing level creation by allowing them to input assets and parameters, and be provided with a pseudo-random map layout, in a format which can be directly implemented in to the game and played by the end user. The maps which this tool will generate will utilise tilemaps and include rooms of varying shapes and sizes, which will all be accessible through at least one other room.

1.2. Objectives

The use of this project is simplifying the creation of world generation in video games for developers, by providing an interface for automatically creating level layouts, without the need for an external program such as Tiled, as this solution will be accessible as a library which can be utilised directly inside the source code of a game. This would reduce the development cost of games by reducing development time and increasing testing speed.

Primary Objectives

- By the end of November, research which procedural generation methods are best fit for this project.

- By the end of the first trimester, have a program which can create a maze-like structure with a start and end point by assembling pre-made room segments.

- By the end of the first trimester, have generation be seed based so results are repeatable.

- By the end of January, have the levels be exportable to a game.

- By the end of February, include automated enemy placement with logical placements, and different spawn conditions based on enemy type.

Secondary Objectives

- By the end of February, include automatically varied room sizes for more variance.

- By the end of February, have full documentation on how best to use the project to avoid user confusion.

- By the end of March, create the ability for the developers to chain multiple maps together, allowing for seamless transitions between different areas in games.

1.3. Scope

This project will produce a playable map layout for games automatically upon loading an instance of the game, this generation will be seed based and include various configurable parameters to change the results. Level generation will be limited to 2D environments that utilise tilemaps.

The bounds of the level generation will be clearly interfaced from the user into the program, as the generation will not be in real time, it will happen during the loading phase of the game, thus the map size cannot be infinite.

1.4. Deliverables

This project will deliver a program, whether that be in the form of a plugin or library, which allows developers to input parameters which determine how a procedurally generated 2D map is generated to be implemented into their game. It will also deliver a detailed report outlining the procedure of creating the project, along with research documentation, and a video demonstration. A document containing a tutorial for how to use the product will also be included on the projects GitHub page.

1.5. Constraints

The deadline for this project is the 29th of April 2027, restricting the project completion time to under a year, limiting what is possible to achieve.

The intended final format of the project being a library limits the capabilities for a user interface as it will be limited to being accessed through an IDE such as visual studio, rather than having its own executable which can use a graphical user interface.

1.6. Assumptions

One assumption is the end user will have useable assets in which the program can turn in to a level layout, however a default texture will be used to avoid errors.
This project also assumes various game features such as enemy movement can react appropriately to different map layouts.

Another assumption is that the user is willing to replace the potential for hand crafted levels with procedurally generated ones, or is willing to spend the extra time customising the generation parameters for multiple levels to achieve a similar effect.

2. Project rationale and operation

2.1. Project benefits

This project aids game developers in level creation by acting as a tool for implementing procedural generation into their games, through several conditional variables which will affect how the level generates. This tool eliminates the need to manually create levels, which would speed up development time for games with multiple levels, and provide a unique experience to each player for games intended to use the same map for a players entire journey. As the project uses a seed to generate its layouts, developers can reuse the same seed multiple times between implementing features to determine exactly how each feature should be generated.

2.2. Project operation

This project will be completed under agile development, where two week sprints will be implemented, until two weeks before the coursework deadline, to ensure consistent progress is made. Agile development also allows for changes to easily be made throughout the project, as documentation is handled in parallel to the creation of the product itself, allowing for more time to research, and make informed changes. Self evaluations will be completed at the end of each sprint to track the effectiveness of this development methodology.

2.3. Options

There are multiple different methods in which the creation and execution of the project can be handled through, these are some options for both.

- There are several different programming languages this project could be completed in, this decision controls which programming languages the project can interact with, lest it is specifically designed to work in all languages.

- This project could be in the form of a plugin for a piece of game development software, there are many options for this such as GoDot, Unity, and Tiled.

- There are many procedural generation techniques that could be used, many of which use a seed. Deciding which to use will have to be based on both speed and customisability, though the potential to add more generation techniques for different features is always an option down the line.

2.4. Risk analysis and mitigation

Risk	Likelihood	Severity	Impact
Bugs in code	5	2	10
End users not knowing how to use the project	2	4	8
Burnout	3	2	6
Loss of data	2	3	6
Minor illness	5	1	5
Major illness	1	4	4

Risk Mitigation

Risk	Mitigation	Likelihood	Severity	Residual Risks
Bugs in code	Apply rigorous testing throughout the project to catch bugs early, along with allocating time towards the end of the project specifically for bug fixes.	2	2	4
End users not knowing how to use the project	Creating a supporting document outlining all of the features and how to use them, updated on the GitHub page as new features are added.	1	1	1
Burnout	Adhering to a strict work schedule which allocates a consistent amount of time to the project throughout its entire duration, avoiding the need for ‘crunch time’.	2	2	4
Loss of Data	Frequently pushing code and documentation to GitHub, along with storing local backups on different drives in case any issues with GitHub arises.	1	1	1
Minor Illness	Structuring the planned work schedule to include a two week empty period at the end of the project, to account for any delays any minor illnesses may	1	1	1

	cause.			
Major Illness	By keeping the documentation up to date with the programming, should any major illnesses arise, a partially complete product will be eligible for submission.	1	3	3

2.5. Resources required

No project specific resources are required, however standard readily available online programming resources such as stack overflow will be utilised. Any downtime on these resources shouldn't have significant impact on the project, as other tasks can be completed while waiting for these delays.

2.6. Ethical and legal considerations

Everything the project achieves will happen client-side, so there is no risk of misuse of user data.

2.7. Commercial considerations

This project will cost £8400 to complete, thus accounting for profit and tax, this project is valued around £13000. This project is aimed at game developers who seek to save time on level generation, while still maintaining a high quality, and have that generation linked directly into their game. As there are many free solutions available already, the customer base will likely be small.

3. Project methodology and outcomes

3.1. Initial project plan

3.1.1. Tasks and milestones

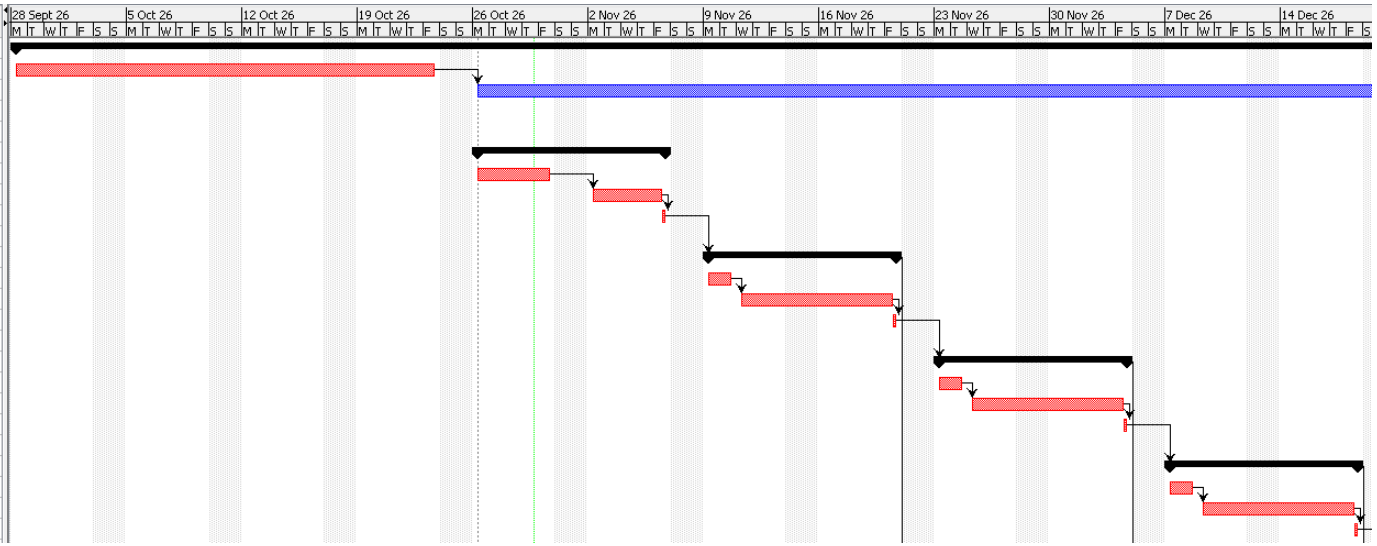
Task	Description	Time Estimate
Implement a seed based procedural generation method which connects rooms with hallways		19 days
Research appropriate procedural generation methods	The method used should generate around a seed.	5 days
Use default assets to automatically generate room layouts connected with paths		7 days
Ensure paths connect in natural ways to both rooms and each other		4 days
Project documentation	Add research findings to the final report and review how successful implementing this feature was.	3 days
Ensure maps are exportable	The maps have to be in a format which can be directly used by games	13.5 days
Research appropriate user interface	Choosing between a library, or plugin for existing program.	4 days
Research different methods of implementing levels into games		2 days
Convert the generated levels into an appropriate format		4 days
Documentation for users	Create a tutorial for how to export maps to their games	0.5 days
Project documentation	Add research findings to the final report and review how successful implementing this feature was.	3 days

Implement customisable variables which alter how levels are generated.		16.5 days
Research methods for variables in generation		3 days
Implement generation variable and allow them to be customisable by the user		10 days
Documentation for users	Create a tutorial for users to follow, providing guidelines on how to generate a map	0.5 days
Project documentation	Add research findings to the final report and review how successful implementing this feature was.	3 days
Automatic enemy placement		16.5 days
Research appropriate methods for implementing procedurally generated enemy locations.		2 days
Allow for users to input several enemy types and how they should be placed.		1 day
Apply a generation technique to each of the enemy types so they are automatically placed on the map		10 days
Documentation for users	Create a tutorial for users to follow for how they can customise enemy spawning.	0.5 days
Project documentation	Add research findings to the final report and review how successful implementing this feature was.	3 days

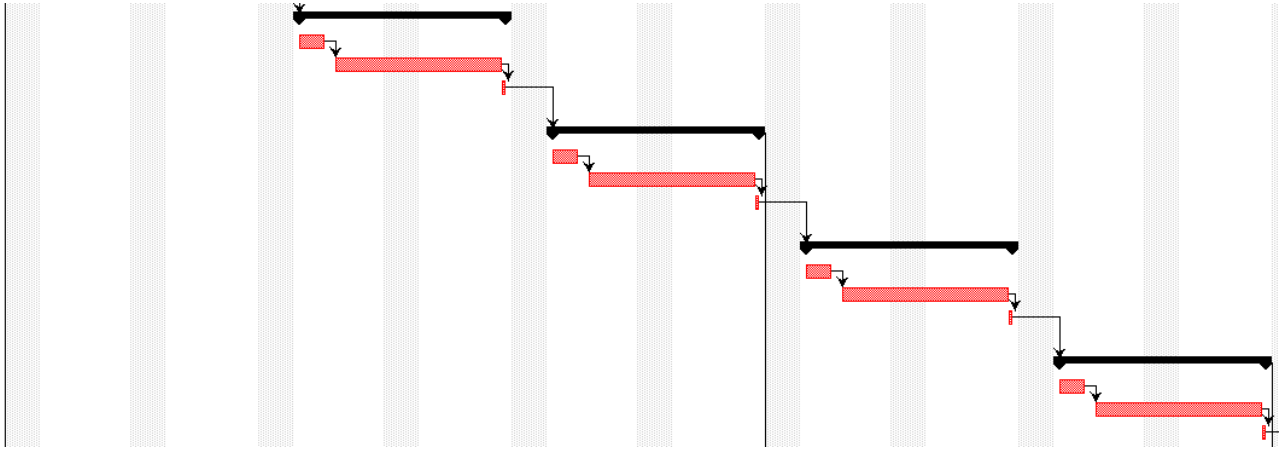
3.2. Schedule Gantt chart

	Name	Duration	Start	Finish
1	**Documentation**	145 days	**28/09/2026, 08:00**	**16/04/2027, 17:00**
2	PDD	20 days	28/09/2026, 08:00	23/10/2026, 17:00
3	Literature Review	55 days	26/10/2026, 08:00	08/01/2027, 17:00
4	Final Documentation	70 days	11/01/2027, 08:00	16/04/2027, 17:00
5	**Sprint #1**	10 days	**26/10/2026, 08:00**	**06/11/2026, 17:00**
6	Research	5 days	26/10/2026, 08:00	30/10/2026, 17:00
7	Programming	4.5 days	02/11/2026, 08:00	06/11/2026, 13:00
8	Evaluation	0.5 days	06/11/2026, 13:00	06/11/2026, 17:00
9	**Sprint #2**	10 days	**09/11/2026, 08:00**	**20/11/2026, 17:00**
10	Research	2 days	09/11/2026, 08:00	10/11/2026, 17:00
11	Programming	7.5 days	11/11/2026, 08:00	20/11/2026, 13:00
12	Evaluation	0.5 days	20/11/2026, 13:00	20/11/2026, 17:00
13	**Sprint #3**	10 days	**23/11/2026, 08:00**	**04/12/2026, 17:00**
14	Research	2 days	23/11/2026, 08:00	24/11/2026, 17:00
15	Programming	7.5 days	25/11/2026, 08:00	04/12/2026, 13:00
16	Evaluation	0.5 days	04/12/2026, 13:00	04/12/2026, 17:00
17	**Sprint #4**	10 days	**07/12/2026, 08:00**	**18/12/2026, 17:00**
18	Research	2 days	07/12/2026, 08:00	08/12/2026, 17:00
19	Programming	7.5 days	09/12/2026, 08:00	18/12/2026, 13:00
20	Evaluation	0.5 days	18/12/2026, 13:00	18/12/2026, 17:00
21	**Sprint #5**	10 days	**04/01/2027, 08:00**	**15/01/2027, 17:00**
22	Research	2 days	04/01/2027, 08:00	05/01/2027, 17:00
23	Programming	7.5 days	06/01/2027, 08:00	15/01/2027, 13:00
24	Evaluation	0.5 days	15/01/2027, 13:00	15/01/2027, 17:00
25	**Sprint #6**	10 days	**18/01/2027, 08:00**	**29/01/2027, 17:00**
26	Research	2 days	18/01/2027, 08:00	19/01/2027, 17:00
27	Programming	7.5 days	20/01/2027, 08:00	29/01/2027, 13:00
28	Evaluation	0.5 days	29/01/2027, 13:00	29/01/2027, 17:00
29	**Sprint #7**	10 days	**01/02/2027, 08:00**	**12/02/2027, 17:00**
30	Research	2 days	01/02/2027, 08:00	02/02/2027, 17:00
31	Programming	7.5 days	03/02/2027, 08:00	12/02/2027, 13:00
32	Evaluation	0.5 days	12/02/2027, 13:00	12/02/2027, 17:00
33	**Sprint #8**	10 days	**15/02/2027, 08:00**	**26/02/2027, 17:00**
34	Research	2 days	15/02/2027, 08:00	16/02/2027, 17:00
35	Programming	7.5 days	17/02/2027, 08:00	26/02/2027, 13:00
36	Evaluation	0.5 days	26/02/2027, 13:00	26/02/2027, 17:00
37	**Sprint #9**	10 days	**01/03/2027, 08:00**	**12/03/2027, 17:00**
38	Research	2 days	01/03/2027, 08:00	02/03/2027, 17:00
39	Programming	7.5 days	03/03/2027, 08:00	12/03/2027, 13:00
40	Evalution	0.5 days	12/03/2027, 13:00	12/03/2027, 17:00
41	**Sprint #10**	10 days	**15/03/2027, 08:00**	**26/03/2027, 17:00**
42	Research	2 days	15/03/2027, 08:00	16/03/2027, 17:00
43	Programming	7.5 days	17/03/2027, 08:00	26/03/2027, 13:00
44	Evaluation	0.5 days	26/03/2027, 13:00	26/03/2027, 17:00
45	Decide on a generation algor	0 days	20/11/2026, 08:00	20/11/2026, 08:00
46	Implement said algorithm	0 days	04/12/2026, 08:00	04/12/2026, 08:00
47	Research end format	0 days	18/12/2026, 08:00	18/12/2026, 08:00
48	Have map be exportable to g	0 days	29/01/2027, 08:00	29/01/2027, 08:00
49	Include randomised enemy p	0 days	26/02/2027, 08:00	26/02/2027, 08:00
50	Implement varied room sizes	0 days	12/03/2027, 08:00	12/03/2027, 08:00
51	Chain multiple levels togethe	0 days	26/03/2027, 08:00	26/03/2027, 08:00

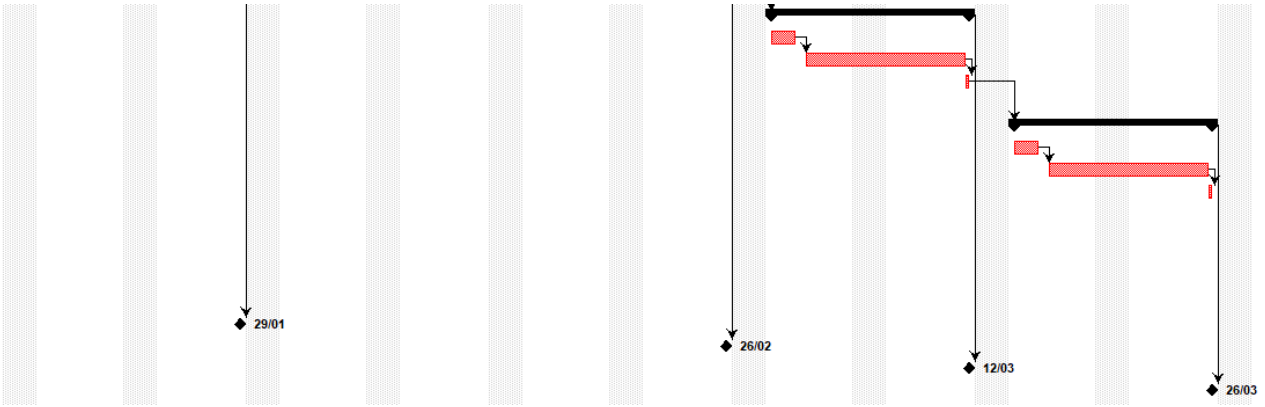
Name	Duration	Start	Finish
Documentation	145 days	**28/09/2026, 08:00**	**16/04/2027, 17:00**
PDD	20 days	28/09/2026, 08:00	23/10/2026, 17:00
Literature Review	55 days	26/10/2026, 08:00	08/01/2027, 17:00
Final Documentation	70 days	11/01/2027, 08:00	16/04/2027, 17:00
Sprint #1	10 days	**26/10/2026, 08:00**	**06/11/2026, 17:00**
Research	5 days	26/10/2026, 08:00	30/10/2026, 17:00
Programming	4.5 days	02/11/2026, 08:00	06/11/2026, 13:00
Evaluation	0.5 days	06/11/2026, 13:00	06/11/2026, 17:00
Sprint #2	10 days	**09/11/2026, 08:00**	**20/11/2026, 17:00**
Research	2 days	09/11/2026, 08:00	10/11/2026, 17:00
Programming	7.5 days	11/11/2026, 08:00	20/11/2026, 13:00
Evaluation	0.5 days	20/11/2026, 13:00	20/11/2026, 17:00
Sprint #3	10 days	**23/11/2026, 08:00**	**04/12/2026, 17:00**
Research	2 days	23/11/2026, 08:00	24/11/2026, 17:00
Programming	7.5 days	25/11/2026, 08:00	04/12/2026, 13:00
Evaluation	0.5 days	04/12/2026, 13:00	04/12/2026, 17:00
Sprint #4	10 days	**07/12/2026, 08:00**	**18/12/2026, 17:00**
Research	2 days	07/12/2026, 08:00	08/12/2026, 17:00
Programming	7.5 days	09/12/2026, 08:00	18/12/2026, 13:00
Evaluation	0.5 days	18/12/2026, 13:00	18/12/2026, 17:00



Sprint #5	10 days	**04/01/2027, 08:00**	**15/01/2027, 17:00**
Research	2 days	04/01/2027, 08:00	05/01/2027, 17:00
Programming	7.5 days	06/01/2027, 08:00	15/01/2027, 13:00
Evaluation	0.5 days	15/01/2027, 13:00	15/01/2027, 17:00
Sprint #6	10 days	**18/01/2027, 08:00**	**29/01/2027, 17:00**
Research	2 days	18/01/2027, 08:00	19/01/2027, 17:00
Programming	7.5 days	20/01/2027, 08:00	29/01/2027, 13:00
Evaluation	0.5 days	29/01/2027, 13:00	29/01/2027, 17:00
Sprint #7	10 days	**01/02/2027, 08:00**	**12/02/2027, 17:00**
Research	2 days	01/02/2027, 08:00	02/02/2027, 17:00
Programming	7.5 days	03/02/2027, 08:00	12/02/2027, 13:00
Evaluation	0.5 days	12/02/2027, 13:00	12/02/2027, 17:00
Sprint #8	10 days	**15/02/2027, 08:00**	**26/02/2027, 17:00**
Research	2 days	15/02/2027, 08:00	16/02/2027, 17:00
Programming	7.5 days	17/02/2027, 08:00	26/02/2027, 13:00
Evaluation	0.5 days	26/02/2027, 13:00	26/02/2027, 17:00



Sprint #9	**10 days**	**01/03/2027, 08:00**	**12/03/2027, 17:00**
Research	2 days	01/03/2027, 08:00	02/03/2027, 17:00
Programming	7.5 days	03/03/2027, 08:00	12/03/2027, 13:00
Evalution	0.5 days	12/03/2027, 13:00	12/03/2027, 17:00
Sprint #10	**10 days**	**15/03/2027, 08:00**	**26/03/2027, 17:00**
Research	2 days	15/03/2027, 08:00	16/03/2027, 17:00
Programming	7.5 days	17/03/2027, 08:00	26/03/2027, 13:00
Evaluation	0.5 days	26/03/2027, 13:00	26/03/2027, 17:00
Decide on a generation algor	0 days	20/11/2026, 17:00	20/11/2026, 17:00
Implement said algorithm	0 days	04/12/2026, 17:00	04/12/2026, 17:00
Research end format	0 days	18/12/2026, 17:00	18/12/2026, 17:00
Have map be exportable to g	0 days	29/01/2027, 17:00	29/01/2027, 17:00
Include randomised enemy p	0 days	26/02/2027, 17:00	26/02/2027, 17:00
Implement varied room sizes	0 days	12/03/2027, 17:00	12/03/2027, 17:00
Chain multiple levels togethe	0 days	26/03/2027, 17:00	26/03/2027, 17:00



3.3. Project control

GitHub will be used for all project management tasks, including handling of the two week sprints via GitHub milestones. It will also be used for version control should any changes need to be reverted, along with automated testing upon each commit to ensure nothing in the project breaks.

3.4. Project evaluation

The project will be considered a success if at least 80% of the objectives are completed, with the bi-weekly sprint evaluations ensuring the continued progress towards this goal.